

Rainier Ordinario - 301631145

Nelson Dang - 301578533

Derek Kang - 301618005

Kenneth Pamintuan - 301582028

CMPT 276 - Phase 2 Report

1. Overall Approach to Implementation

The game is a grid-based Java application in which a prisoner navigates four increasingly complex prison sections: Cell Block, Cafeteria/Washrooms, General Time, and Outdoor Recreation, and must escape before a per-level time limit expires. The player collects coins to unlock each section's exit, avoids cops, and picks up bonus food items to restore health.

The implementation follows a layered architecture. The GameEngine class acts as the central coordinator, delegating all state to specialised components: Board owns the grid of Cell objects; ScoreManager tracks score and coin collection; Clock tracks time elapsed; and the entity hierarchy (Player, Cop) models everything that moves on the board. GameUI is a pure view built on Java Swing that reads state from GameEngine on every frame but never writes to it.

Each of the four levels is defined by a LevelConfig object produced by a dedicated factory class. This separation ensures that level-specific data such as start and exit coordinates, required coins, and cop spawn positions is isolated from game loop logic, making it straightforward to add or modify levels without touching GameEngine.

2. Adjustments and Modifications to the Initial Design

Compared to the initial Phase 1 design, several changes were made during implementation.

Food now restores HP. In Phase 1, food was simply a collectible item. It was changed to actively restore the player's health when collected, giving it a more meaningful role in gameplay.

Gold was renamed to coins. This was a design choice as coins serve the same purpose as gold in the original design, acting as the currency needed to unlock level exits.

Item is not abstract. The UML shows Item as an abstract class with an abstract applyEffect(). In the implemented code, Item is a concrete base class with added name and itemType fields. This was necessary because coin pickups are represented directly as Item objects, which do not need to override applyEffect(). Subclasses RegularItem and BonusRewards still override behaviour as planned.

Cop movement methods were refined. The UML specifies a single `moveTowardPlayer()` method on Cop. The actual implementation splits this into `backAndForth()` for patrol mode and `chasePlayer()` for pursuit mode, coordinated by a `seesPlayer()` vision check. This two-mode AI more accurately models a guard that patrols until it detects the player and then pursues.

Factory Method design pattern added. The Phase 1 design did not include a level-creation mechanism. Four concrete factory classes, `CellBlockFactory`, `CafeteriaFactory`, `GeneralTimeFactory`, and `OutdoorRecFactory`, extending an abstract `LevelFactory` were introduced. Each factory produces an immutable `LevelConfig` value object, cleanly encapsulating level-specific configuration.

`CorruptibleCop` mechanic disabled. The `CorruptibleCop` class is present in the codebase as designed in Phase 1. However, after playtesting, the bribe mechanic was found to disrupt the intended challenge balance. The corruptible cop is no longer spawned in any level's factory configuration and the bribe key binding was removed from the UI. The class remains intact for potential reactivation.

`ScoreManager` extended for coin tracking. The Phase 1 `ScoreManager` only included score-related methods. Fields `coinsCollected` and `coinsRequired` along with methods `addCoin()`, `resetCoins()`, and `hasEnoughCoins()` were added. Coin logic is contained entirely within `ScoreManager`; `GameEngine` only calls `scoreManager.hasEnoughCoins()` to gate level exits.

`GameUI` added. The Phase 1 UML focused on game logic only. A complete Swing-based `GameUI` class was implemented as the graphical front-end, featuring pixel-art sprites for the player and cops, fog-of-war rendering, and a HUD showing time, health, and coin status.

Regarding use cases, the core use cases from Phase 1 remained largely intact. The Collect Items use case was updated to reflect that coins specifically must be collected to unlock the exit. The Escape Jail use case was updated to require that the player has collected enough coins before the exit becomes accessible. The Bribe Cop use case remains in the design but is currently inactive as the `CorruptibleCop` is not spawned in any level.

3. Management Process and Division of Roles

The project was divided into four primary roles. Rainier was responsible for `GameEngine`, level flow, the `LevelFactory` hierarchy, and the `GameUI` graphical front-end. Derek was responsible for the Board and Cell system, including the map file parser that reads `.txt` level files and populates the grid. Kenneth was responsible for all entities, player movement, and cop AI including patrol and chase behaviour. Nelson was responsible for items, rewards, `ScoreManager`, `Clock`, and `Trap`.

The team planned two milestones for the construction phase. By the midway deadline, the goal was to have all core logic classes implemented and compiled, including Board, Cell, Trap, all entity classes, ScoreManager, Clock, and items, with GameEngine in a working skeleton state. By the final deadline, the goal was to have GameEngine fully wired to all components, all four levels functional, the GameUI complete, and the game fully playable from start to finish. Both milestones were met on schedule.

With this plan established, the team worked in parallel on their respective sections and integrated continuously as classes became available. When a teammate's class compiled, dependent roles could begin wiring against it. The team held regular group calls to discuss errors and integration blockers, which helped reduce large merge conflicts and kept progress consistent. When one role needed a class owned by another teammate, it was flagged to the group immediately so the class could be stubbed or completed as quickly as possible.

4. External Libraries

javax.swing (Java Standard Library). The GUI is built entirely on Java Swing, part of the Java SE standard library, requiring no additional dependencies in pom.xml. Swing was chosen because it is included in every Java SE distribution, the project targets Java 17 which fully supports Swing, and the grid-based game renders well with JPanel.paintComponent() and manual Graphics2D drawing. The team also had prior Java experience but limited familiarity with frameworks such as JavaFX or libGDX, making Swing the lowest-risk choice within the project timeline.

JUnit 5 Jupiter (test scope only). Unit testing uses JUnit 5, declared in pom.xml at test scope. JUnit 5 was chosen because it is the standard modern Java testing framework and is supported natively by Maven Surefire without additional configuration.

No other external libraries were used. All game logic, data structures, and utility functions rely entirely on the Java SE standard library.

5. Measures Taken to Enhance Code Quality

JavaDoc on all public members. Every class, field, and method has a `/** */` JavaDoc comment describing its purpose, parameters, and return values, ensuring the codebase is self-documenting and readable by all team members.

BFS-verified level coordinates. All cop spawn positions, player start positions, and exit coordinates were validated using a breadth-first search reachability analysis. Positions that fell inside walls were corrected and coordinates unreachable from the start position were relocated to reachable cells. A runtime fallback, `GameEngine.findNearestWalkable()`, performs BFS outward from any desired spawn

position to correct it to the nearest walkable cell, making the game robust against future map edits.

Immutable value objects. LevelConfig is immutable as all collections are defensively copied on construction and in getters, preventing accidental mutation of level configuration data during gameplay.

Single-responsibility separation. Coin logic lives entirely in ScoreManager. Level configuration lives entirely in factory classes. Rendering logic lives entirely in GameUI. GameEngine coordinates these components but owns none of their internal state.

Fog-of-war as a pure visual effect. The fog of war is implemented exclusively in GamePanel's drawing methods using a single isFoggy() helper. No game logic in GameEngine was modified, respecting the separation between view and logic.

Agile-style incremental development. The team first implemented the core logic matching the UML diagram, then refactored and adjusted the code before implementing the GUI. This ensured the underlying architecture was stable before the view layer was built on top of it.

6. Biggest Challenges

Map correctness and cops spawning inside walls. The original level map files contained coordinates that fell inside wall tiles, causing entities to clip into walls and breaking movement. The solution was to analyse each level map with a BFS reachability check to identify valid walkable positions and correct all factory coordinates. A runtime fallback was also added as a safety net.

Level 3 start position trapped in a dead-end. Level 3's original start position was inside an isolated pocket with no connection to the main map. BFS confirmed the pocket was completely disconnected from the reachable area. The start position was relocated to a cell connected to the full map.

Collaboration and merge conflicts. For most team members this was their first experience collaborating on a shared codebase. The team encountered numerous merge conflicts, accidental overwrites of working code, duplicate files such as two Clock.java files in different folders, and inconsistent file naming. These were resolved through regular group communication, clearly divided ownership of each class, and prompt reporting when a blocking dependency was needed.

Balancing coursework and availability. All team members were balancing this project alongside other courses, part-time or full-time work, and club commitments. Finding time to meet as a group was difficult. Despite this, the team maintained consistent progress through regular check-ins and a shared commitment to completing their assigned sections on time.